Can you create a drafter website template with a dataclass called state with attributes Displacement:list[Distance] Velocity_current:float Momentum_current:float Net_force:float and a second dataclass called Calculations with attributes Distance:float Time:float with routes called index, calculate_momentum Update_velocity Update_distance Update_time Create_graph Show_method. Make a page for each route that lets the user input their answer and compares it to calculated data from the euler cromer method. If the answers are not equal they will be directed to a page that tells them they got the wrong answer. The euler cromer method is momentum_final = momentum_initial + (Net_force * time), Final_velocity = initial_velocity + (momentum_final/time), Final_distance = initial_distance + (Final_velocity * time), Time_final = time_initial + time. After the user goes through all the steps, they will see a graph created by the code looping through the euler cromer method with the input data that will show the trajectory of the particle. They will then continue to the last page that can show the full process of the euler cromer method. They can then click a button to start over at the original page. There should be a button on every page to move the user to the next page.

Worked for 35s

Absolutely! We can build this as a multi-page Drafter Euler–Cromer Simulator using Python, dataclasses, and separate routes.

I'll structure it so the user works through the physics calculations one step at a time. At each step, the website will:

1. Display the information needed for the calculation.
2. Allow the user to enter their answer.
3. Compare their answer with the Euler–Cromer calculation.
4. Direct them to a wrong-answer page if their answer is incorrect.
5. Allow them to proceed to the next calculation if correct.

At the end, the website will generate a trajectory graph and display the complete calculation process.

1. Website structure

The routes will follow this order:

index

Enter initial conditions

calculate_momentum

Calculate final momentum

Update_velocity

Calculate final velocity

Update_distance

Calculate final displacement

Update_time

Calculate final time

Create_graph

Display particle trajectory

Show_method

Display the complete Euler–Cromer process

There will also be a `wrong_answer` route that the user can visit whenever an answer is incorrect.

2. Important physics clarification

There is one correction to make to the equations before implementing the program.

For the Euler–Cromer method:

- Momentum:
- $pf=pi+F\Delta t$
- p
- f
-
- $=p$
- i

-
- $+F\Delta t$
- Velocity:
- vf=pf/m
- v
- f
-
- $=p$
- f
-
- $/m$
- Displacement:
- xf=xi+vfΔt
- x
- f
-
- $=x$
- i
-
- $+v$
- f
-
- Δt
- Time:
- tf=ti+Δt
- t
- f
-
- $=t$
- i
-
- $+\Delta t$

Your proposed velocity equation uses `momentum_final / time`. To calculate velocity from momentum, we need to divide by mass, not time.

I'll use mass as an input and keep `delta_t` constant for each iteration.

3. Dataclasses and State

I recommend keeping your distance and time values in two separate lists, as you previously preferred. Their matching values will always have the same index.

For example:

Index	Displacement	Time
0	0.0	0.0
1	2.5	0.5
2	6.0	1.0
3	10.5	1.5

I'll use `Distance = float` so you can retain your requested `list[Distance]` type.

The `State` dataclass also needs a few extra attributes to remember the mass, time interval, number of iterations, and previous calculations.

4. Complete Drafter starter template

This is a complete starting implementation. It uses Drafter's `@route`, `Page`, `TextBox`, and `Button` components, along with Matplotlib for the graph. These are supported Drafter features.

Drafter Documentation
+2

You can copy this into your Python file.

```
        state.velocity_history.append(expected)
```

```
        return Update_distance(state)

    return Page(state, [
        Header("Step 2: Calculate Velocity"),
        "Final momentum: " + str(state.momentum_current),
        "Mass: " + str(state.mass),
        "Calculate final velocity:",
        "v final = p final / mass",
        TextBox("answer"),
        Button("Check velocity", "Update_velocity")
    ])



# --------------------------------
# Step 3: Displacement
# --------------------------------

@route
def Update_distance(state: State,
                    answer: float = None) -> Page:

    expected = (state.displacement[-1]
                + state.velocity_current * state.delta_t)

    if answer is not None:
        if not math.isclose(answer, expected,
                            rel_tol=1e-6, abs_tol=1e-6):
            state.expected_answer = expected
            state.submitted_answer = answer
            state.retry_route = "Update_distance"
            return wrong_answer(state)

        state.displacement.append(expected)
        return Update_time(state)

    return Page(state, [
        Header("Step 3: Calculate Displacement"),
        "Initial displacement: " + str(state.displacement[-1]),
```

```
        "Final velocity: " + str(state.velocity_current),
        "Time interval: " + str(state.delta_t),
        "Calculate final displacement:",
        "x final = x initial + (v final * delta t)",
        TextBox("answer"),
        Button("Check displacement", "Update_distance")
    ])


# --------------------------------
# Step 4: Time
# --------------------------------

@route
def Update_time(state: State,
                answer: float = None) -> Page:

    expected = state.times[-1] + state.delta_t

    if answer is not None:
        if not math.isclose(answer, expected,
                            rel_tol=1e-6, abs_tol=1e-6):
            state.expected_answer = expected
            state.submitted_answer = answer
            state.retry_route = "Update_time"
            return wrong_answer(state)

        state.times.append(expected)

        # Save matching distance and time values
        state.calculations.append(
            Calculations(
                distance=state.displacement[-1],
```

5. How the looping works

The most important part of this project is making sure the website can repeat the four calculations for however many iterations the user entered.

Here is the logic:

```
# Each iteration follows this sequence:

momentum_final = momentum_current + net_force * delta_t

velocity_final = momentum_final / mass

distance_final = displacement[-1] + velocity_final * delta_t

time_final = times[-1] + delta_t

# Save the matching values
displacement.append(distance_final)
times.append(time_final)
```

Notice the use of `[-1]`. This accesses the most recent value in a list.

For example, `displacement[-1]` gets the previous displacement, allowing the next calculation to build on it.

The website then checks:

```
if state.current_step < state.total_steps:
   return calculate_momentum(state)
else:
   return Create_graph(state)
```

This is what allows the simulation to repeat without creating a separate route for every iteration.

6. One improvement for your Show_method page

I recommend displaying the actual substitutions into the equations, not just the final answers.

For example, you could replace the `process.append(...)` section with:

```
process.append(
   "Iteration " + str(i + 1) + "\n"
   + "Momentum: "
   + str(state.momentum_history[i])
   + " + (" + str(state.net_force)
   + " * " + str(state.delta_t)
   + ") = " + str(state.momentum_history[i + 1])
   + "\nVelocity: "
   + str(state.momentum_history[i + 1])
   + " / " + str(state.mass)
   + " = " + str(state.velocity_history[i + 1])
   + "\nDisplacement: "
   + str(state.displacement[i])
   + " + (" + str(state.velocity_history[i + 1])
   + " * " + str(state.delta_t)
   + ") = " + str(state.displacement[i + 1])
   + "\nTime: "
   + str(state.times[i])
   + " + " + str(state.delta_t)
   + " = " + str(state.times[i + 1])
)
```

This will show the user exactly how each answer was calculated.

7. Important things to test

Before considering the website finished, test these situations:

- Enter normal initial conditions and complete one iteration.
- Complete multiple iterations.
- Enter an incorrect momentum answer.
- Enter an incorrect velocity answer.
- Enter an incorrect displacement answer.
- Enter an incorrect time answer.
- Check that time and displacement values stay aligned.
- Check that the graph contains every iteration.
- Check that Start Over allows a new simulation.

My biggest recommendation: Build and test the first iteration before worrying about the graph or the final explanation page. Once the four calculation routes work correctly, the rest of the website becomes much easier to build.

You can use the official

Drafter documentation

and

MatPlotLibPlot reference

if you need to check component syntax.